Beyond Shapley: Efficient Computation of Assymetric Shapley Values

Ezequiel Companeetz[1], Santiago Cifuentes[2], Sergio Abriola[2]

[1] Departamento de Computación, Facultad de Ciencias Exactas y Naturales, UBA
[2] Institudo de Ciencias de la Computación (ICC), Conicet

Abstract. We address the problem of explainability in machine learning models through feature attribution methods. In particular, we consider a variant of Shapley values known as Asymmetric Shapley Values (ASV), which enables the incorporation of causal knowledge into model-agnostic explanations through the use of a causal graph. We show that in certain contexts in which the computation of SHAP is #P-hard, the exact computation of ASV can be done in polynomial time. To extend this algorithmic result, we introduce a notion of equivalence classes over the topological orderings of the underlying causal graph, which is useful to reduce the time to compute ASV. In particular, we present a polynomial-time algorithm (in the number of equivalence classes) to compute it whenever the causal graph is a rooted directed tree. Finally, we develop an algorithm for approximating ASV in arbitrary causal DAGs which relies on a procedure to sample topological orderings uniformly at random. To implement this sampling mechanism we leverage known algorithms as well as simpler alternatives. Our experimental result demonstrate the practical viability of the proposed approach in realistic causal structures.

Keywords: Explicability, Asymmetric Shapley Values (ASV), Shap, Topological ordering

Más allá de Shapley: Algoritmos Eficientes para calcular los Shapley Values Asimétricos

Abstract. En este artículo se aborda el problema de la explicabilidad en modelos de aprendizaje automático mediante métodos de feature attribution. En particular, se estudia una variante de los Shapley values conocida como Asymmetric Shapley Values (ASV), que permite incorporar conocimiento causal en la explicación de modelos de forma model-agnostic. A partir del análisis de su complejidad, se demuestra que el cálculo exacto de ASV es polinomial en modelos cuya distribución de entrada está representada por una red bayesiana del tipo Naive Bayes, en contraste con SHAP, que es #P-hard aún en este caso restringido.

Con el objetivo de extender estos resultados a clases más generales de redes bayesianas, se introduce una noción de clases de equivalencia sobre los órdenes topológicos del grafo causal subyacente, lo cual permite reducir drásticamente el número de permutaciones necesarias para computar ASV. Se presenta un algoritmo polinomial en el número de clases para identificarlas, y se implementa un esquema de cómputo exacto de ASV basado en estas clases. Además, se propone un nuevo método para computar en tiempo polinomial la predicción esperada de un árbol de decisión, sobre una distribución dada por una red bayesiana arbitraria, permitiendo así evaluar el algoritmo desarrollado para el cómputo de ASV en estos modelos. Por último, se propone un algoritmo aproximado para calcular el ASV en familias de DAG's causales del tipo polytree. Para ello, se desarrolla un algoritmo de muestreo aleatorio de órdenes topológicos de polytrees. Estos resultados respaldan la viabilidad del enfoque propuesto en estructuras causales realistas, y se contrastan empíricamente con SHAP tanto en precisión como en eficiencia computacional.

Keywords: Explicabilidad, Asymmetric Shapley Values (ASV), Shap, Órdenes topológicos

1 Introduction

In recent years, the capabilities of artificial intelligence models have grown rapidly, along with the complexity of their architectures. This progress has enabled the solution of tasks that once seemed unattainable, such as solving complex mathematical problems using language models (et. al, 2024). At the same time, current systems have reached scales far beyond those of a few years ago, both in terms of the number of parameters and structural complexity (Epoch AI, 2024). This lack of transparency is particularly problematic in sensitive contexts, such as medical diagnosis or high-stakes decision-making, where obtaining a correct prediction is not sufficient: it is also crucial to understand how that prediction was reached. In this context, the field of Explainable Artificial Intelligence (XAI) has emerged, with the goal of developing methods that can explain model decisions.

Within the field of explainability, one of the most influential approaches is feature attribution, where each input variable is assigned a score reflecting its contribution to the prediction. In this line of work, SHAP is one of the most well-known methods (Lundberg & Lee, 2017). It is based on the Shapley values, a concept originating in cooperative game theory (Shapley et al., 1953). Intuitively, Shapley values provide a fair way to distribute the total value achieved by a coalition among its participants, according to their average marginal contribution when joining. Translated to machine learning, features play the role of

players, and the value assigned to each feature aims to quantify its contribution to the model's prediction for a given instance.

Despite its popularity, SHAP has important limitations. On the one hand, depending on the family of ML models under consideration and the underlying distribution of the data, the computation of the Shapley values can be tractable or not. For example, it is well-known that for product distributions the complexity of computing these scores is equivalent to the complexity of computing the average value of the model (Van den Broeck et al., 2022). In particular, this implies that there exist efficient algorithms for the case where the models are given, for example, by decision trees or by restricted families of circuits (Arenas et al., 2023), while the problem is intractable whenever the models are more expressive, as in the case of neural nets. For more general families of distributions the problem becomes much harder: for instance, when considering Naive Bayes distributions, the computation of this score is intractable already when considering trivial models that only depend on one feature (Van den Broeck et al., 2022).

On the other hand, there are also concerns about the quality of the explanations it produces. In particular, it is not always clear how to interpret, in the context of AI models, the axioms inherited from game theory, and several works point out that these explanations may fail to adequately capture relationships such as dependence, relevance, or causality between variables (Fryer et al., 2021; Huang & Marques-Silva, 2023; Marques-Silva, 2023). More generally, there is no absolute consensus on what should be considered an "explanation" in AI: many current techniques are pragmatic approximations to a much broader problem related to understanding, causality, and human interpretability (Lipton, 2017; Miller, 2019).

The Asymmetric Shapley Values (ASV from now on) are a proposal introduced by Frye et al., 2019 to overcome these limitations. This variant incorporates causal information into the computation of attribution scores by restricting attention to permutations of variables that respect a causal graph. In this way, instead of treating all features symmetrically, priority is given to the more fine-grained ones (in the sense that the information they provide is not present in the other variables). This can produce explanations that are more faithful to the data-generating process and can help better detect phenomena such as bias or discrimination. Moreover, although ASV was primarily proposed to improve explanatory quality, it naturally raises the question of whether this additional restriction may also provide computational benefits.

In this paper we address therefore the problem of efficiently computing the ASV scores, which was not explored thoroughly in previous literature. We will prove that for some families of models for which computing the Shapley values is intractable there are still polynomial time algorithms for computing the ASV. Inspired by this initial result, we develop an algorithm to compute the ASV exactly in more general contexts by exploiting the fact that many permutations of the features can be grouped in an unique equivalence class to simplify computations. Finally, we also propose a flexible approximation scheme that allows to

approximate the ASV as long as it is possible to (1) Sample a topological order of the causal graph uniformly at random, and (2) Sample a data point with respect to the data distribution after conditioning a subset of the features to have specific values. We instantiate all these general algorithms with particular examples of causal graphs and ML models and show experimentally that they are efficient.

2 Definitions

Let X be a finite set of features. A (binary) *entity* over X is a mapping $e : X \to \{0, 1\}$ assigning a value to every feature from X. We denote by $\texttt{ent}(X)$ the set of all entities, and we will assume that some probability distribution is provided over this set. A (binary) classifier is a mapping $M : \texttt{ent}(X) \to \{0, 1\}$. We restrict to both binary entities and classifiers only for simplicity: all the results and algorithms we obtain can be extended to more general cases straightforwardly.

Given a model M and an entity e, a *feature attribution score* is a mapping $\phi_{M,e} : X \to \mathbb{R}$. Intuitively, the value $\phi_{M,e}(x)$ indicates the relevance of feature x with respect to prediction $M(e)$. As mentioned before, one of the most studied feature attribution schemes is the SHAP-SCORE (Lundberg & Lee, 2017), which is theoretically grounded on the *Shapley values* (Shapley et al., 1953) from cooperative game theory. In that context, the Shapley values come up as the "most fair" way to distribute some capital earned during a team game. More precisely, given a finite set of players $\mathcal{I}$ and a *characteristic function* $\nu : \mathcal{P}(\mathcal{I}) \to \mathbb{R}$ describing, for each subset of the players, the reward that such coalition would obtain in some fixed game; the shapley values are the unique assignment $\varphi_\nu : \mathcal{I} \to \mathbb{R}$ of reward to each player i satisfying the following desired properties:

1. Efficiency: All the reward is distributed, i.e. $\sum_{i \in \mathcal{I}} \varphi_\nu(i) = \nu(\mathcal{I})$.
2. Symmetry: If $\nu(S \cup \{i\}) = \nu(S \cup \{j\})$ for every $S \subseteq \mathcal{I}$, then $\varphi_\nu(i) = \varphi_\nu(j)$.
3. Null Player: If $\nu(S \cup \{i\}) = 0$ for every $S \subseteq \mathcal{I}$, then $\varphi_\nu(i) = 0$.
4. Linearity: For any pair of characteristic functions ν_1, ν_2 and $a \in \mathbb{R}$ it holds that $\varphi_{a\nu_1+\nu_2}(i) = a\varphi_{\nu_1}(i) + \varphi_{\nu_2}(i)$[3].

Moreover, the Shapley values have an exact closed form: if we refer to players with indices in the range $\{1, \ldots, n\}$, we can write

$$\varphi_\nu(i) = \frac{1}{|\mathcal{I}|!} \sum_{\pi \in perm(\mathcal{I})} [\nu(\pi_{<i} \cup \{i\}) - \nu(\pi_{<i})] \tag{1}$$

where $perm(\mathcal{I})$ denotes the set of all permutations of the number from 1 to n and $\pi_{<i}$ denotes the set of indices before i in the permutation π (i.e. $\pi_{<i} = \{j : \pi(j) < i\}$). Fundamentally, Eq. (1) is summing across all possible ways of ordering the features, considering the impact of the inclusion of player i to the reward of the game when adding the players according to each of these orderings.

[3] The characteristic function $a\nu_1 + \nu_2$ is naturally defined as $(a\nu_1 + \nu_2)(S) = a\nu_1(S) + \nu_2(S)$.

In the context of machine learning explainability, the Shapley values are instantiated by thinking of the features as players and considering the characteristic function

$$\nu_{M,e}(S) = \mathbb{E}[M(e')|S, e] = \sum_{e' \in \mathtt{ent}(X)} M(e') \Pr[e'|\{e'(x) = e(x) : x \in S\}] \tag{2}$$

which represents the expected value of the model M when restricting the features from S to have the value dictated by entity e.[4]

As mentioned in the introduction, the Shapley values may fail to distinguish features that are heavily correlated. For example, if some feature j depends deterministically of some other feature i it can happen that both features receive the same score, even though i is intuitively more relevant for *any* model. Note that the *Symmetry* property essentially implies that this should always happen whenever two features share the same information relevant for a prediction. Thus, to avoid this problem it is somewhat necessary to relax that axiom and therefore define a different score. This is the idea underlying the *Assymetric Shapley Values*.

Definition 1 (Assymetric Shapley Values (Frye et al., 2019)). *Given a model M, an entity e, a feature x and some weight function $w : perm(X) \to \mathbb{R}_{\geq 0}$ satisfying*

$$\sum_{\pi \in perm(X)} w(\pi) = 1,$$

we define the Assymetric Shapley value of x as

$$\phi^{ASV}_{M,e}(x) = \sum_{\pi \in perm(X)} w(\pi) \left[\nu_{M,e}(\pi_{<i} \cup \{i\}) - \nu_{M,e}(\pi_{<i})\right]$$

where $\nu_{M,e}$ is defined in Eq. (2)

This definition is derived from the corresponding notion of asymmetric shapley values from cooperative game theory. The function w is useful for weighting some orderings above others. For example, if some feature j is a function of i it is possible to introduce this fact into the score by assigning weight zero to all orderings in which i comes after j. Note that the Shapley values are obtained by taking $w(\pi) = \frac{1}{|X|!}$, and we will refer to this particular case as $\phi^{Shap}_{M,e}$.

Frye et al., 2019 proposes to define the weight function w by considering a *causal DAG* C over the features. More precisely, C is a directed acyclic graph whose nodes are the set of features X and where an edge directed from i to j indicates that j depends on i. Then, they define the weight function

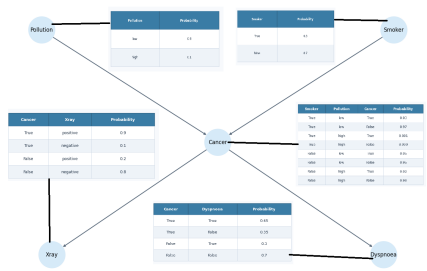
$$w_C(\pi) = \begin{cases} \frac{1}{|topos(C)|} & \pi \text{ is a topological sort of } C \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

[4] In some proofs we may write the characteristic function as $\nu_{M,e,\Pr}$ to make the dependence on the distribution Pr explicit.

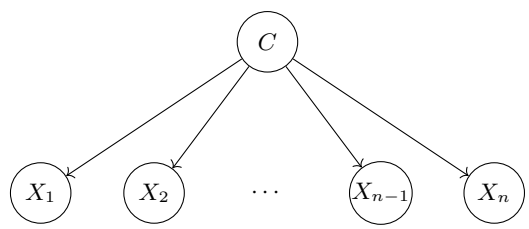
This weight function generalizes the idea mentioned earlier of considering only those orderings consistent with the known causality of the features. In the rest of this paper we will be interested in developing efficient algorithms for computing the ASV from Def. 1 whenever the weight function is given by Eq. (3).

Moreover, we will consider that the distribution of space of entities is described by a Bayesian Network N. We refer to the reader to standard bibliography for the precise formal definitions of Bayesian Networks (Pearl, 1986) while here we describe them conceptually: a bayesian network N defines a distribution for $\texttt{ent}(X)$ with a DAG whose nodes are the features from X. Every node x has a set of parents $p(x) \subseteq X$, and the network provides, for each way of assigning the features in $p(x)$, the conditional distribution of x. See Figure 1a for an example. Given N and any entity e it is possible to compute the probability of e by traversing the DAG of N in any topological sort multiplying the corresponding conditional probabilities.

One of the most simplest types of Bayesian Networks is the Naive Bayes, which was already mentioned in regards to intractability results for the Shapley values. This network has a particular structure, in which a single node is parent to all the others and no other pairs of nodes are connected. See Figure 1b for an example.



(a) Example of a cancer prediction Bayesian network with its CPTs.



(b) A Naive Bayes network.

Fig. 1: Examples of Bayesian network structures.

A key point for our purposes is that computing the characteristic function under a Bayesian network distribution requires probabilistic inference. Since exact inference in Bayesian networks is #P-HARD in general Cooper, 1990, we focus on restricted structures such as polytrees, where inference can be performed in polynomial time.

Sometimes we will identify the network itself as a causal graph. Thus, the network will provide both the distribution necessary to compute the characteristic function $\nu_{M,e}$ as well as the causal graph necessary to define the weight function w^C.

This identification is not always appropriate. In general, a Bayesian network represents a joint probability distribution and its conditional independencies,

but its directed edges do not necessarily encode causal relations. Therefore, using the same graph as both a probabilistic model and a causal graph is only justified when the network is assumed to reflect the underlying causal structure. Otherwise, this should be understood merely as a modeling simplification.

3 Theoretical Results

3.1 Tractability of ASV

As mentioned before, the computation of the Shapley Values is intractable (in particular, #P-hard[5]) even for trivial models if the underlying distribution is represented by a Naive Bayes network. We now show that this is not the case for the ASV whenever we use as causal graph the bayesian network itself.

Theorem 1. *Let $\mathcal{F}$ be any family of models. Then, the ASV can be computed in polynomial time for the family $\mathcal{F}$ under Naive Bayes distributions (where the causal graph is the network itself) if and only if the Shapley values can be computed in polynomial time for the family $\mathcal{F}$ under product distributions*[6].

Proof. First, we show the right-to-left implication, which is the hard one. Let x_1 be the root node of the causal graph, and let $x_2, \ldots, x_n$ be the rest of the nodes. The DAG has $(n-1)!$ topological sorts, and in particular it holds that, for any $2 \leq j \leq n$,

$$\phi^{ASV}_{M,e,\Pr}(x_j) = \frac{1}{(n-1)!} \sum_{\pi \in perm(\{x_2,\ldots,x_n\})} [\nu_{M,e,\Pr}(\{x_1\} \cup \pi_{<j} \cup \{x_j\}) - \nu_{M,e,\Pr}(\{x_1\} \cup \pi_{<j})]$$

Thus, once x_1 is fixed the distribution over $x_2, \ldots, x_n$ is a product distribution with probabilities $\Pr(x_j = 1 | x_1 = e(x_1))$ for $2 \leq i \leq n$. For simplicity, let's assume $e(x_1) = 1$. Then, if we define the product distribution $\Pr'$ given by

$$\Pr'[X_i = 1] = \begin{cases} 1 & i = 1 \\ \Pr[X_i = 1 | X_1 = e(x_1)] & \text{otherwise} \end{cases}$$

it is easy to see that for any $S \subseteq X \setminus \{x_1\}$ it holds that

$$\nu_{M,e,\Pr}(\{x_1\} \cup S) = \nu_{M,e,\Pr'}(S) \tag{4}$$

This is intuitive: the distribution $\Pr'$ is obtained by fixing the value of x_1 as $e(x_1)$, and thus having x_1 inside ν does not matter.

[5] #P is a class of *counting* problems. The #P-hard problems are the hardest problems in this class, and it is known that if they can be solved in polynomial time then P = NP. Then, under the usual theoretical assumption, #P-hard problems cannot be solved efficiently.

[6] With our notation, a product distribution over $\texttt{ent}(X)$ is a distribution that factorizes as $\Pr[e] = \prod_{x \in X} \Pr[x = e(x)]$.

From Eq. (4) it can be proven easily that

$$\phi^{ASV}_{M,e,\Pr}(x_j) = \phi^{Shap}_{M,e,\Pr'}(x_j) \tag{5}$$

which implies that we effectively reduced the problem of computing the asymmetric Shapley values for any feature x_j with $j \geq 2$ to computing the Shapley values for some product distribution. Finally, for x_1 it can be seen that

$$\phi^{ASV}_{M,e,\Pr}(x_1) = \nu_{M,e,\Pr}(\{x_1\}) - \nu_{M,e,\Pr}(\emptyset)$$

because $\pi_{<1} = \emptyset$ for any permutation consistent with the DAG. Computing the values $\nu_{M,e,\Pr}(\{x_1\})$ and $\nu_{M,e,\Pr}(\emptyset)$ is equivalent to computing averages of the model M, and it is known that the capacity of computing the Shapley values for any product distribution entails the capacity of computing averages (Van den Broeck et al., 2022).

The left-to-right implication is immediate by observing that product distributions are a particular case of Naive Bayes distribution, and thus we can "simulate" them: one possible way to do this is to consider, for each $1 \leq i \leq n$, the Naive Bayes distribution $\Pr_i$ given by placing x_i as parent node with no conditioning on its children (i.e. $\Pr_i[x_j = 1|x_i = 0] = \Pr_i[x_j = 1|x_i = 1]$). Then, it holds that

$$\phi^{Shap}_{M,e,\Pr}(x_j) = \frac{1}{n}\sum_{i=1}^{n} \phi^{ASV}_{M,e,\Pr_i}(x_j)$$

Theorem 1 shows that, in contrast to the Shapley values, the ASV can be computed under Naive Bayes distributions for simple models such as decision trees. Intuitively, the reason for this is that the causality imposed by the distribution is mimicked by the way that only certain permutations are considered, and thus many symmetries can be exploited. We believe this motivates the idea of studying the complexity of computing the ASV whenever the distribution is also used as the causal graph.

3.2 Equivalence classes of topological sorts

The Shapley values can also be written in closed form as

$$\phi^{Shap}_{M,e}(x) = \sum_{S \subseteq X \setminus \{x\}} \frac{|S|!(|X| - |S| - 1)!}{|X|!} \left[\nu_{M,e}(S \cup \{x\}) - \nu_{M,e}(S)\right]$$

This rewriting is obtained by observing that many permutations give raise to the same set $\pi_{<x}$: more precisely, if $|\pi_{<x}| = s$ there are $\frac{s!(n-s-1)!}{n!}$ permutations π' satisfying $\pi'_{<x} = \pi_{<x}$, where $n = |X|$. Computing the Shapley values using this formula reduces the number of times that $\nu_{M,e}$ has to be computed, although the number of calls remains exponential on the number of features.

We will follow a similar strategy to compute the ASV. We define a notion of equivalence between permutations.

Definition 2 (Equivalence relation for permutations). *Let C be a causal graph for features X and fix some feature $x \in X$. We define an equivalence relation $\sim_x$ over $topos(C)$ as*

$$\pi^1 \sim_x \pi^2 \iff \{\pi^1_{<x}\} = \{\pi^2_{<x}\}.$$

Let Λ^x_C be the set of equivalence classes. It is immediate from the definition that

$$\phi^{ASV}_{M,e}(x) = \frac{1}{|topos(C)|} \sum_{[\pi]\in\Lambda^x_C} |[\pi]| \left[\nu_{M,e}(\pi_{<x} \cup \{x\}) - \nu_{M,e}(\pi_{<x})\right] \tag{6}$$

Therefore, instead of invoking $\nu_{M,e}$ $|topos(C)|$ times it is enough to compute it $|\Lambda^x_C|$ times. This value can be at most $2^{|X|}$, but the hope is that in practical scenarios it is smaller. From now on, we will restrict to the case in which causal graphs are given by *rooted directed trees*, polytrees with nodes that have one or less parents. In this case, note that we can split the nodes in three groups with respect to x: those that are descendants of x, those that are ancestors of x, and the *unrelated* ones. Observe that descendant/ancestors are never/always in the set $\pi_{<x}$, for any x. Thus, the structure of the equivalence classes depends only on the unrelated nodes, you can see them in Figure 2.

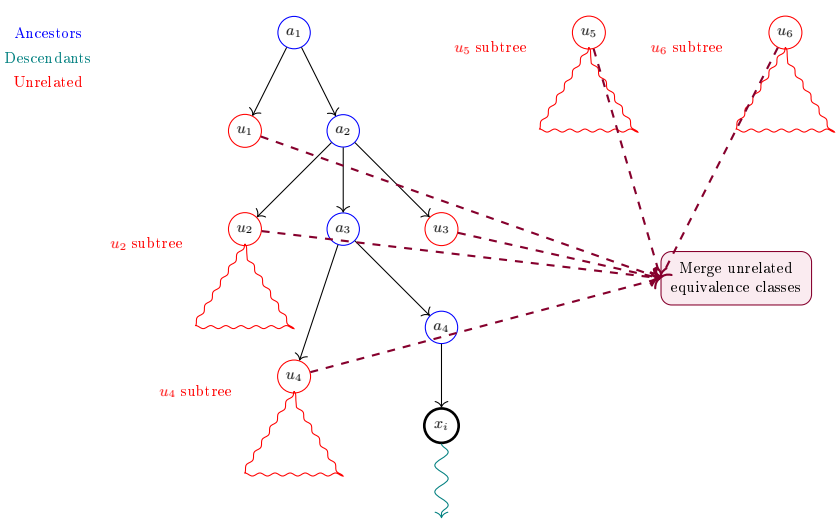


Fig. 2: Graph partition into ancestors, descendants and unrelated nodes respect to x_i, the feature for which we want to calculate the ASV. The unrelated rooted subtrees can be processed independently and then merged to construct the full equivalence classes.

We have a general bound on the size of Λ^x_C depending only on the causal graph C:

Proposition 1. *Let C be a causal graph shaped as a rooted directed tree with h leaves and depth l. Then, it holds that*

$$|\Lambda_C^x| \leq h^l + 1$$

Proof. For every unrelated feature y (in the sense that y is neither a descendant or ancestor of x) let $C_y \subseteq C$ be the directed tree rooted at y induced by those nodes reachable by y. Let $f(y)$ denote the number of ways that the features in C_y can be arranged in the permutations, i.e.

$$f(y) = |\{S \subseteq C_y : \exists \pi \in perm(C) \text{ such that } \pi_{<x} \cap C_y = S\}| \tag{7}$$

Note that

$$f(y) = \begin{cases} 2 & y \text{ is a leaf} \\ 1 + \prod_{z \text{ child of } y} f(z) & \text{otherwise} \end{cases} \tag{8}$$

In the base case the set of Eq. (7) is $\{\emptyset, \{y\}\}$. For the recursive case the reasoning is the following: either we do not put y in the set (and then all other features reachable from y won't be in the set as well) or rather we put y and have complete freedom to decide which features to put from the sub-trees of y. The bound from the statement of the proposition can be derived using Eq. (8).

Proposition 1 indicates that the number of equivalence classes will be small when the tree is shallow. From the point of view of the causal graph, this corresponds to the case in which there aren't long causal dependencies between the features. In particular, there will be many features that are mutually independent. In our experimentation we used balanced trees, multiple path graphs, Naive Bayes and other classes of graphs which don't have a very long causal dependencies.

For the case of directed trees we further develop an algorithm able to compute the set Λ_C^x as well as the number of permutations inside each class (which is required to compute Eq. (6)). The complexity of the algorithm is polynomial on $|\Lambda_C^x|$.

Proposition 2. *There is an algorithm able to compute $|[\pi]_R|$ given the causal graph C (shaped as a rooted directed tree) on time polynomial on $|C|$ and $|\Lambda_C^x|$. Moreover, the algorithm returns the size of each class $[\pi] \in \Lambda_C^x$.*

Echu: Ofrezco dos opciones, una más por arriba y otra que entra más en detalle. Choose your posion. PD: No puse nada de la complejidad pero no se cómo poner algo sin que sea humo, porque no estoy explicando nada.

Proof. The algorithm is recursive on its nature, and based on the ideas from the proof of Proposition 1.

OPCION 1: First we recursively compute the equivalence classes of each unrelated subtree. We then combine these classes with one another, and finally integrate them with the ancestors and descendants while respecting the precedence constraints imposed by the DAG. In particular, descendants are added as

a block on the right of x_i, while ancestors require counting the valid interleavings with the nodes placed to its left. This yields all equivalence classes and their sizes without explicitly enumerating topological orders.

OPCION 2: We first focus on the unrelated part: for each subtree rooted at a node in UR, we recursively compute its equivalence classes. This is done by combining the classes of its children and deciding whether each node appears to the left or to the right of x_i, which fully determines its contribution. As a result, each unrelated subtree can be processed independently, yielding a set of partial equivalence classes together with their associated counts.

We then combine these classes across all unrelated subtrees and integrate them with the ancestors and descendants. Descendants are always placed to the right of x_i and can therefore be treated as a block, whose contribution is given by the number of topological orders of the corresponding subtree. In contrast, ancestors impose precedence constraints, so we must count the valid interleavings between them and the nodes placed to the left. Altogether, this construction yields all equivalence classes and their sizes without explicitly enumerating topological orders.

We note that once the set Λ_C^x is computed it is possible to find any value $\phi_{M,e}^{ASV}(x)$ in time linear on $|\Lambda_C^x|$. Thus, even though this step can be costly (because the bound from Proposition 1 implies that the algorithm may take exponential time on the size causal graph) is must be done only once. Moreover, it is independent of the model M, and thus it can be reused even when the AI model is updated.

3.3 Approximating ASV through sampling

A simple way to compute $\phi_{M,e}^{Shap}(x)$ consist on sampling a small subset $K \subseteq perm(X)$ at random and then approximating the score as

$$\phi_{M,e}^{Shap}(x) \sim \frac{1}{|K|} \sum_{\pi \in K} [\nu_{M,e}(\pi_{<x} \cup \{x\}) - \nu_{M,e}(\pi_{<x})]$$

For the ASV the analogous procedure consists on sampling topological orderings at random. In the next statement we formalize this strategy. Moreover, we observe that the algorithm is robust to error both at the level of the sampling procedure and the computation of $\nu_{M,e}$.

Proposition 3. *Assume access to a procedure $\mathcal{P}_{samp}$ that samples topological orderings from a DAG with a distribution ε_{samp}-close to the uniform one*[7]*, and to a procedure $\mathcal{P}_\nu$ that computes, given any subset of features $S \subseteq X$, a ε_ν-approximation of the value $\nu_{M,e}(S)$. Then, it is possible to compute, with high probability, a $O(\varepsilon_{samp} + \varepsilon_\nu)$-approximation of $\phi_{M,e}^{ASV}(x)$ using $O(1/\varepsilon_{samp}^2)$ calls to $\mathcal{P}_{samp}$ and $\mathcal{P}_\nu$.*

[7] To measure distance between distributions we use the 1-norm, i.e. given two distributions $\mathcal{D}_1$ and $\mathcal{D}_2$ over a finite domain Ω we define the distance between $\mathcal{D}_1$ and $\mathcal{D}_2$ as $||\mathcal{D}_1 - \mathcal{D}_2||_1 = \sum_{o \in \Omega} |\mathcal{D}_1(o) - \mathcal{D}_2(o)|$.

Proof. Consider the random variable $V^{M,e}$ over domain $topos(C)$ defined as $V^{M,e}(\pi) = [\nu_{M,e}(\pi_{<x} \cup \{x\}) - \nu_{M,e}(\pi_{<x})]$. It holds that

$$\mathbb{E}[V^{M,e}] = \phi^{ASV}_{M,e}(x)$$

Now, if $V^{M,e}_{\varepsilon_{samp}}$ denotes the analogous random variable distributed according to the sampling procedure $\mathcal{P}_{samp}$,

$$\begin{aligned}
&|\mathbb{E}[V^{M,e}] - \mathbb{E}[V^{M,e}_{\varepsilon_{samp}}]| = \\
&= \left| \sum_{\pi \in topos(C)} \left[\Pr\left(V^{M,e} = \pi\right) - \Pr\left(V^{M,e}_{\varepsilon_{samp}} = \pi\right) \right] [\nu_{M,e}(\pi_{<x} \cup \{x\}) - \nu_{M,e}(\pi_{<x})] \right| \\
&\leq \sum_{\pi \in topos(C)} \left| \Pr\left(V^{M,e} = \pi\right) - \Pr\left(V^{M,e}_{\varepsilon_{samp}} = \pi\right) \right| \leq \varepsilon_{samp}
\end{aligned}$$

Then, to approximate $\mathbb{E}[V^{M,e}_{\varepsilon_{samp}}]$ we apply the procedure $\mathcal{P}_{samp}$ to compute a subset K of the topological orderings and then use $\mathcal{P}_\nu$ to obtain

$$\mathbb{E}[V^{M,e}_{\varepsilon_{samp}}] \sim \frac{1}{|K|} \sum_{\pi \in K} [\mathcal{P}_\nu(\pi_{<x} \cup \{x\}) - \mathcal{P}_\nu(\pi_{<x})]$$

Using Hoeffding's inequality (**hoeffding1963probability**) we can ensure that by taking $O(1/\varepsilon^2_{samp})$ samples the set K is a good representation of the average with high probability (i.e. with error bounded by ε_{samp}), and then using the ε_ν-closeness of $\mathcal{P}_\nu$ we can show that the error obtained by approximating all the $\nu_{M,e}$ invocations is bounded by $O(\varepsilon_\nu)$.

Proposition 3 states that we only need to be able to sample the topological orderings *almost* uniformly to compute the ASV. Also, it shows that if we approximate $\nu_{M,e}$ the overall *additive*-error of the estimation of $\phi^{ASV}_{M,e}(x)$ is also controlled. Note that implementing the procedure $\mathcal{P}_\nu$ is often easy: by definition $\nu_{M,e}(S)$ is the expected value of a random variable, and thus if we can sample from the set of entities after conditioning that $s = e(s)$ for every $s \in S$ we are done. This is the case for the product distribution, but also for some bayesian networks such with an underlying graph which is a polytree, because the inference can be done in polynomial time.

Now we describe two ways to implement the procedure $\mathcal{P}_{samp}$. The first one can be understood by looking at Figure 3. The idea is that we can sample a topological ordering uniformly using a recursive a procedure in which at each step we pick the left-most node of the ordering. In particular, in the beginning we must pick one of the sources of the causal graph (i.e. one of the nodes with in-degree 0). We should weight them according to the number of topological orderings that start with each respective node, and thus we effectively reduce the problem of sampling a topological ordering to the problem of counting the number of topological orderings. Even though this problem is intractable in general Brightwell and Winkler, 1991, there are heuristic algorithms that can

approximate this value or even exact algorithm for restricted cases Huber, 2006; Kangas et al., 2016.

In particular, we develop a simple algorithm applicable to polytrees whose complexity depends on the maximum degree of a node.

Echu: Acá también dejo dos opciones, la opción más por arriba y la opción más descriptiva.

OPCION 1: This algorithm computes the number of topological orderings of a polytree by performing a DFS over its underlying undirected graph and recursively combining the results of each node's unvisited neighbors. Instead of only counting the number of orderings of each subtree, the algorithm keeps track of the positions in which a node may appear, together with how many topological orderings realize each position. This additional information is necessary to correctly handle nodes with multiple parents, where simple multinomial interleavings are no longer sufficient. At each step, the algorithm enumerates the valid relative orders of the current node and its unvisited neighbors, combines the recursive results of those neighbors, and merges equal positions. In this way, it computes all possible placements of the node and the corresponding number of topological orderings, yielding an exact counting algorithm for arbitrary polytrees.

OPCION 2:

The algorithm computes the number of topological orderings of a polytree by performing a DFS over its underlying graph and recursively combining the results of each node's unvisited neighbors. At each step, it considers all valid relative orders of the node and its neighbors, combines their contributions, and merges equal positions. This yields an exact counting algorithm for polytrees without explicitly enumerating all topological orders.

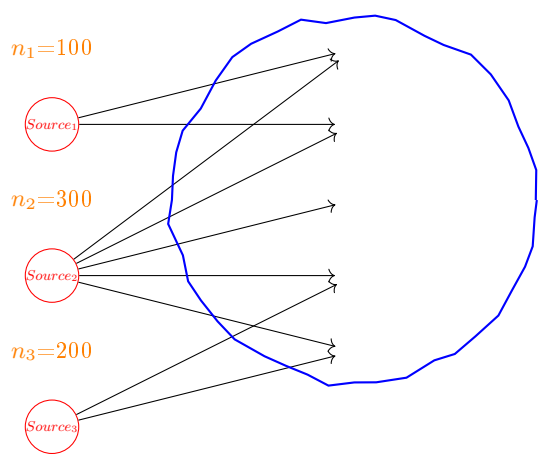


Fig. 3: Illustration of the first step of the sampling algorithm. The candidate source nodes that may appear first in the ordering are shown in red, while the rest of the graph is shown in blue. Each source node is annotated with the number of topological orderings in which it appears first.

Alternatively, we consider known algorithms from the literature able to approximate uniform sampling of topological orderings from arbitrary DAGs. We implement an algorithm based on random walks (**bubley1999faster**) with a polynomial (although of high degree) complexity. We note that there are more modern and asymptotically better algorithms (**huber2006fast**), but we believe that the hidden constants in them make them inefficient for the input sizes we consider.

4 Experimental results

4.1 Experimental Evaluation

We evaluate the proposed methods by comparing them against naive baselines in terms of computational cost and scalability. Experiments were conducted on two Bayesian networks, *Cancer* and *Child*, obtained from the bnlearn repository. Both were used as data distributions and as causal DAGs. The *Child* was modified into a polytree by removing edges that introduce cycles in the underlying undirected graph.

Datasets were generated by sampling from these networks, and decision trees were trained accordingly. For ASV computation, the prediction variable was removed from the network to avoid trivial inference.

Experimental setup

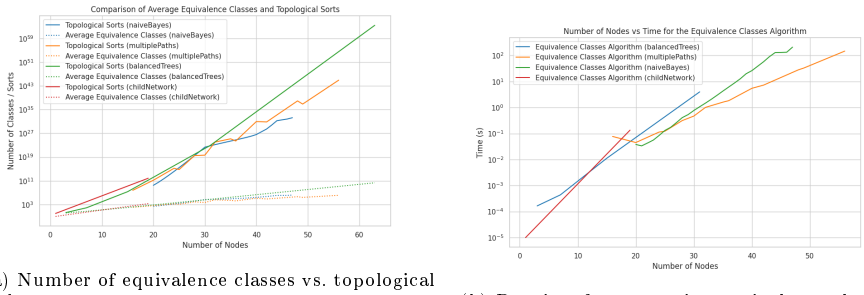
- CPU: Intel i5-7500 @ 3.40GHz
- RAM: 16 GB
- OS: Ubuntu 22.04
- Python 3.12
- Libraries: pgmpy, sklearn, networkx, shap

Equivalence Classes vs. Topological Orders We compare the naive ASV computation based on all topological orders with our approach based on equivalence classes. The key quantities are the sizes of $topos(G)$ and $eqCl(G, x_i)$, as well as the cost of constructing each set.

Figure 4 shows that the number of equivalence classes grows significantly slower than the number of topological orders. For instance, in *Child*, the graph has 7.41×10^{11} topological orders but only around 2003 equivalence classes on average. Similar gaps are observed in balanced trees, where the difference can reach several orders of magnitude.

At the same time, the cost of computing equivalence classes remains moderate for graphs of practical size, typically under a few seconds. For example, in *Child*, all equivalence classes are computed in less than one second.

This reduction directly decreases the number of evaluations of the characteristic function, making the equivalence class approach substantially more efficient.



(a) Number of equivalence classes vs. topological orders.

(b) Runtime for computing equivalence classes.

Fig. 4: Comparison between equivalence classes and topological orders in terms of size and computation time.

Performance of Equivalence Class Algorithms We now analyze the cost of computing equivalence classes.

As shown in Figure 4b, the computation remains efficient for moderate graph sizes, typically under a few seconds. For example, in *Child*, all equivalence classes are computed in less than one second. However, as graph size increases, the runtime grows alongside the number of equivalence classes.

Figure 5 highlights the main advantage of our approach. While enumerating all topological orders quickly becomes infeasible (e.g., requiring days for large graphs), computing equivalence classes remains tractable. This is because the number of equivalence classes depends on the graph structure rather than on the total number of topological orders.

Sampling-Based Approximation of Topological Orders For graphs where equivalence classes are not directly available, we evaluate the sampling-based approach.

Number of sampled orders	**Sampling time (s)**
100	0.9270
1000	1.1170
10000	4.7170
20000	7.7170
30000	10.8387

Table 1: Sampling times for topological orders in *Child*.

Table 1 shows that sampling remains efficient even for large numbers of samples. Moreover, the runtime is not linear in the number of samples, as multiple candidates are generated simultaneously during recursive calls.

Finally, we evaluate the approximation error of ASV using sampled orders.

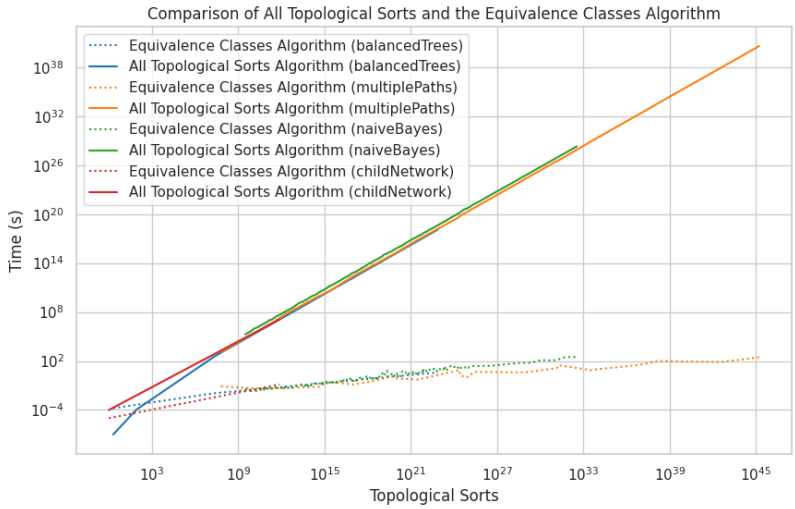


Fig. 5: Runtime comparison between computing equivalence classes and enumerating all topological orders.

Figure 6 shows that using 1000 samples yields an average error of around 8%. Larger ASV values exhibit smaller relative error, while larger deviations correspond to very small true values.

These results indicate that sampling provides a practical trade-off between accuracy and computational cost.

Remarks on Mean Prediction Computation In our experiments, the cost of evaluating the characteristic function is not the dominant factor. For decision trees under Bayesian Networks distributions with an underlying polytree, mean prediction can be computed efficiently, and thus does not limit the performance of the overall approach. Also we compared our algorithm to the brute force approach and it was a 100 times faster, you can see the experiments in the code repository.

5 Conclusions

Disclosure of Interests. for example: The authors have no competing interests to declare that are relevant to the content of this article.

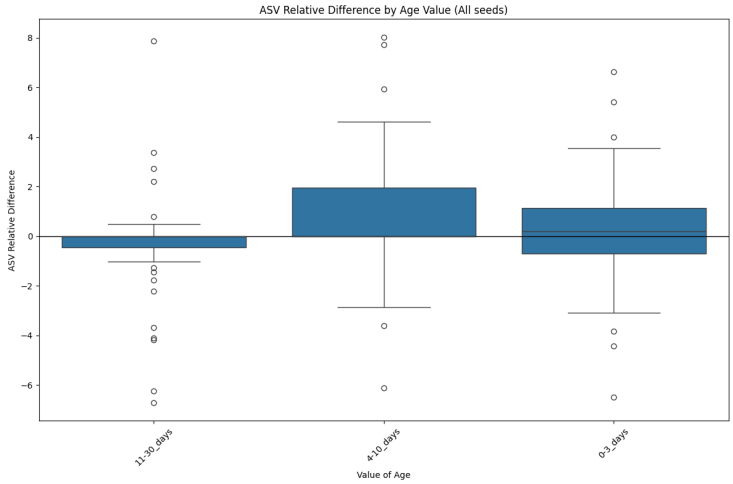


Fig. 6: Relative error between approximate and exact ASV values.

References

Arenas, M., Barceló, P., Bertossi, L., & Monet, M. (2023). On the complexity of Shap-score-based explanations: Tractability via knowledge compilation and non-approximability results. *Journal of Machine Learning Research*, *24*(63), 1–58.

Brightwell, G. R., & Winkler, P. (1991). Counting linear extensions. *Order*, *8*, 225–242. https://doi.org/10.1007/BF00383444

Cooper, G. F. (1990). The computational complexity of probabilistic inference using bayesian belief networks. *Artificial Intelligence*, *42*(2–3), 393–405. https://doi.org/10.1016/0004-3702(90)90060-D

Epoch AI. (2024, June). Data on large-scale Ai models [Accessed: 2025-04-14]. https://epoch.ai/data/large-scale-ai-models

et. al, E. G. (2024). Frontiermath: A benchmark for evaluating advanced mathematical reasoning in Ai. https://arxiv.org/abs/2411.04872

Frye, C., Rowat, C., & Feige, I. (2019). Asymmetric Shapley values: Incorporating causal knowledge into model-agnostic explainability. *arXiv preprint arXiv:1910.06358*.

Fryer, D., Strümke, I., & Nguyen, H. (2021). Shapley values for feature selection: The good, the bad, and the axioms. *Ieee Access*, *9*, 144352–144360.

Huang, X., & Marques-Silva, J. (2023). The inadequacy of Shapley values for explainability. *arXiv preprint arXiv:2302.08160*.

Huber, M. (2006). Fast perfect sampling from linear extensions. *Discrete Mathematics*, *306*(4), 420–428. https://doi.org/https://doi.org/10.1016/j.disc.2006.01.003

Kangas, K., Hankala, T., Niinimäki, T., & Koivisto, M. (2016). Counting linear extensions of sparse posets. *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence*, 603–609.

Lipton, Z. C. (2017). The mythos of model interpretability. https://arxiv.org/abs/1606.03490

Lundberg, S., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. https://arxiv.org/abs/1705.07874

Marques-Silva, J. (2023). Logic-based explainability in machine learning. In *Reasoning web. causality, explanations and declarative knowledge: 18th international summer school 2022, berlin, germany, september 27–30, 2022, tutorial lectures* (pp. 24–104). Springer.

Miller, T. (2019). Explanation in Artificial intelligence: Insights from the social sciences. *Artificial Intelligence*, *267*, 1–38. https://doi.org/https://doi.org/10.1016/j.artint.2018.07.007

Pearl, J. (1986). Fusion, propagation, and structuring in belief networks. *Artificial Intelligence*, *29*(3), 241–288.

Shapley, L. S., et al. (1953). A value for n-person games. *Classics in game theory*.

Van den Broeck, G., Lykov, A., Schleich, M., & Suciu, D. (2022). On the tractability of Shap explanations. *Journal of Artificial Intelligence Research*, *74*, 851–886.